

Week 11

Task1:

To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components

```
#include <stdio.h>
#include <stdlib.h>
int getInput(){
    int n;
    printf("Enter a positive number: ");
    if (scanf("%d", &n) != 1){
        printf("Error: Invalid input.\n");
        exit(1);
    }
    return n;
}
int validateInput(int n){
    if (n <= 0){
        printf("Error: Number must be positive.\n");
        return 0;
    }
    return 1;
}
int calculateSquare(int n)
{
    return n * n;
}
void displayResult(int result){
    printf("Square = %d\n", result);
}
int main(){
    int number, result;
    printf("=== Module Integration Demo ===\n");
    number = getInput();
    if (!validateInput(number)){
        printf("Program terminated due to invalid input.\n");
        return 1;
    }
    result = calculateSquare(number);
    displayResult(result);
    printf("Execution completed successfully.\n");
    printf("Reliability check passed.\n");
    printf("Components successfully integrated.\n");
    return 0;
}
```

Output:

```
root@DESKTOP-VMN932K:~# ./connect
=== Module Integration Demo ===
Enter a positive number: 2
Square = 4
Execution completed successfully.
Reliability check passed.
Components successfully integrated.
root@DESKTOP-VMN932K:~#
```

Task2:

To Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
FILE *logFile;
void logError(const char *msg){
    logFile = fopen("error.log", "a");
    if (logFile != NULL){
        fprintf(logFile, "ERROR: %s\n", msg);
        fclose(logFile);
    }
}
int divide(int a, int b){
    if (b == 0){
        printf("Runtime Error: Division by zero!\n");
        logError("Division by zero.");
        return 0;
    }
    return a / b;
}
int main(){
    int a, b;
    printf("=== Runtime Error Handling Demo ===\n");
    printf("Enter two integers: ");
    if (scanf("%d %d", &a, &b) != 2){
        printf("Syntax Error: Invalid input.\n");
        logError("Invalid input syntax.");
        return 1;
    }
    int result = divide(a, b);
    if (b != 0)
        printf("Result = %d\n", result);
    printf("Program recovered gracefully.\n");
    printf("Error scenarios tested successfully.\n");
    return 0;
}

```

Output:

```

root@DESKTOP-VMN932K:~# ./runtime
=== Runtime Error Handling Demo ===
Enter two integers: 20 5
Result = 4
Program recovered gracefully.
Error scenarios tested successfully.
root@DESKTOP-VMN932K:~#

```

Task3:

To Run Valgrind, Identify Leaks, Fix Memory Errors,
Release Resources, Verify Cleanup, Document Findings

```

GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>

int main(){
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    if (n <= 0){
        printf("Invalid size.\n");
        return 1;
    }
    int *arr = (int *)malloc(n * sizeof(int));
    if (arr == NULL){
        printf("Memory allocation failed.\n");
        return 1;
    }
    printf("Enter %d integers:\n", n);
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);
    printf("Elements are:\n");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
    free(arr);
    arr = NULL;
    printf("Memory released successfully.\n");
    return 0;
}

```

Output:

```

root@DESKTOP-VMN932K:~# ./valgrind
Enter number of elements: 2
Enter 2 integers:
20 5
Elements are:
20 5
Memory released successfully.
root@DESKTOP-VMN932K:~#

```

Task4:

To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports

```

#include <stdio.h>
typedef struct{
    int input1;
    int input2;
    int expected;
} TestCase;
int add(int a, int b){
    return a + b;
}
int main(){
    TestCase tests[] = {
        {2, 3, 5},
        {10, 20, 30},
        {-5, 5, 0},
        {100, 200, 300},
        {7, 8, 16}
    };
    int total = sizeof(tests) / sizeof(tests[0]);
    int pass = 0, fail = 0;
    FILE *report = fopen("test_report.txt", "w");
    if (report == NULL){
        printf("Unable to create report file.\n");
        return 1;
    }
    fprintf(report, "==== Test Report =====\n\n");
    printf("Running Test Cases...\n\n");
    for (int i = 0; i < total; i++){
        int actual = add(tests[i].input1, tests[i].input2);
        if (actual == tests[i].expected){
            printf("Test %d : PASS\n", i + 1);
            fprintf(report, "Test %d : PASS\n", i + 1);
            pass++;
        }
        else{
            printf("Test %d : FAIL\n", i + 1);
            printf("Expected = %d, Actual = %d\n",
                tests[i].expected, actual);
            fprintf(report,
                "Test %d : FAIL (Expected=%d, Actual=%d)\n",
                i + 1,

```

```

                i + 1,
                tests[i].expected,
                actual);
            fail++;
        }
    }
    fprintf(report, "Total Tests : %d\n", total);
    fprintf(report, "Passed      : %d\n", pass);
    fprintf(report, "Failed      : %d\n", fail);
    fclose(report);
    printf("\n==== Summary =====\n");
    printf("Total Tests : %d\n", total);
    printf("Passed      : %d\n", pass);
    printf("Failed      : %d\n", fail);
    printf("\nReport generated: test_report.txt\n");
    return 0;
}

```

Output:

```

root@DESKTOP-VMN932K:~# ./testscript
Running Test Cases...

Test 1 : PASS
Test 2 : PASS
Test 3 : PASS
Test 4 : PASS
Test 5 : FAIL
Expected = 16, Actual = 15

==== Summary =====
Total Tests : 5
Passed      : 4
Failed      : 1

Report generated: test_report.txt
root@DESKTOP-VMN932K:~#

```